LAPORAN TUGAS PEMROGRAMAN PARALEL

Penjumlahan Matriks



Dosen Pengampu:

Amir Murtako, S.Kom., M.Kom

Nama:

Nayunda Krisna / 4524210077

PROGRAM STUDI TEKNIK INFORMATIKA

UNIVERSITAS PANCASILA

2026

Source Code:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <omp.h>
#include <cstdlib>
#include <ctime>

void fillMatrix(std::vector<std::vector<int>>& mat, int size) {
    for (int i = 0; i < size; ++i) {
        for (int j = 0; j < size; ++j) {
            mat[i][j] = rand() % 10 + 1;
        }
    }
}

void matrixAddSerial(const std::vector<std::vector<int>>& A,
    const std::vector<std::vector<int>>& B,
    std::vector<std::vector<int>>& C, int size) {
        for (int i = 0; i < size; ++i) {
            for (int j = 0; j < size; ++j) {
                C[i][j] = A[i][j] + B[i][j];
            }
        }
    }

void matrixAddParallel(const std::vector<std::vector<int>>& A,
    const std::vector<std::vector<int>>& B,
    std::vector<std::vector<int>>& C, int size) {
        #pragma omp parallel for collapse(2)
        for (int i = 0; i < size; ++i) {
            for (int j = 0; j < size; ++j) {
                C[i][j] = A[i][j] + B[i][j];
            }
        }
    }

void runBenchmark(int size) {
    std::cout << "=========================================" <<
std::endl;
```

```
    std::cout << "         UKURAN MATRIKS: " << size << "x" <<
size << std::endl;
    std::cout << "=========================================" <<
std::endl;

    std::vector<std::vector<int>> A(size,
std::vector<int>(size));
    std::vector<std::vector<int>> B(size,
std::vector<int>(size));
    std::vector<std::vector<int>> C_serial(size,
std::vector<int>(size));
    std::vector<std::vector<int>> C_parallel(size,
std::vector<int>(size));

    fillMatrix(A, size);
    fillMatrix(B, size);

    auto start_time = std::chrono::high_resolution_clock::now();
    matrixAddSerial(A, B, C_serial, size);
    auto end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> serial_dur = end_time -
start_time;

    start_time = std::chrono::high_resolution_clock::now();
    matrixAddParallel(A, B, C_parallel, size);
    end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> parallel_dur = end_time -
start_time;

    std::cout << "Serial Duration   : " << serial_dur.count() <<
" seconds" << std::endl;
    std::cout << "Parallel Duration : " << parallel_dur.count()
<< " seconds" << std::endl;
    std::cout << "Speedup           : " << serial_dur.count() /
parallel_dur.count() << "x" << std::endl;
}

int main() {
    srand(time(0));

    std::cout << "---------------------------------" <<
std::endl;
```

```
    std::cout << "NAMA : NAYUNDA KRISNA ABI PASYA" << std::endl;
    std::cout << "NPM  : 4524210077" << std::endl;
    std::cout << "---------------------------------" <<
std::endl;

    int threads = omp_get_max_threads();
    std::cout << "Number of threads: " << threads << std::endl;

    runBenchmark(50);

    runBenchmark(500);

    std::cout << "\n========================================="
<< std::endl;

    return 0;
}
```

Hasil Compile:

```
PS C:\Users\Nayunda Krisna\Documents\1 UniVerse\Semester 5\Pemrograman Paralel\2> g++ -fopenmp -O2 penjumlahan_matriks.cpp -o penjumlahan_matriks.exe
PS C:\Users\Nayunda Krisna\Documents\1 UniVerse\Semester 5\Pemrograman Paralel\2> .\penjumlahan_matriks.exe
---------------------------------
NAMA : NAYUNDA KRISNA ABI PASYA
NPM  : 4524210077
---------------------------------
Number of threads: 16
=======================================
        UKURAN MATRIKS: 50x50
=======================================
Serial Duration   : 0 seconds
Parallel Duration : 0.003019 seconds
Speedup           : 0x
=======================================
        UKURAN MATRIKS: 500x500
=======================================
Serial Duration   : 0 seconds
Parallel Duration : 0 seconds
Speedup           : nanx

=======================================
```